O código faz o que a fórmula diz?

os testes das funções do robô, com dados e gráficos

Material de estudo para os autores do TG (não faz parte do texto)

30 de outubro de 2026

Em uma página

A pergunta. As funções em MQL5 do robô (MQO, ADF, Ljung-Box, verossimilhança e AIC, Hannan-Rissanen, GARCH, previsão) calculam de fato o que as fórmulas dos livros dizem?

Como se respondeu. Com dois tipos de teste, todos num programa só (`ferramentas/TestesFormulas.mq5`), que chama as *mesmas* funções que o robô usa.

- **Identidade**: a conta é refeita do zero, escrita à parte a partir da fórmula, e comparada com o número que o código do robô dá. Tem de dar igual, a menos do arredondamento do computador.
- **Simulação**: geram-se milhares de séries com propriedades conhecidas (por exemplo, um passeio aleatório, em que o ADF *não* deveria rejeitar a raiz unitária) e confere-se se o código se comporta como a teoria prevê.

O resultado. As 20 identidades passaram. Onze deram diferença exatamente zero, e das outras a maior foi $3{,}1 \times 10^{-14}$, no nível do arredondamento do computador (a do valor crítico do ADF é um caso à parte, explicado na seção 4). Nas simulações, os testes estatísticos rejeitam perto de 5% quando deveriam rejeitar 5%, os coeficientes conhecidos são recuperados e o GARCH acha o máximo da verossimilhança.

O que apareceu de novo (seções 5 e 8). O Ljung-Box com o desconto de graus de liberdade rejeita 7,9% nos resíduos de um ARMA(1,1) ajustado, acima dos 5%. E com 240 observações, que é a janela do robô, o β do GARCH sai com muita incerteza. Nenhum dos dois é erro de código, mas ambos são bons de saber.

Sumário

1 Como ler este relatório **2**

1.1 O que é “dar igual” num computador . . . 2

1.2 O que é “dar perto” numa simulação . . . 2

1.3 De onde vêm os dados reais . . . 2

2 As identidades, todas de uma vez **3**

3 Mínimos quadrados (`Numerico.mqh`) **3**

4 O teste ADF (`Estatistica.mqh`, `SelecaoModelo.mqh`) **4**

4.1 A conta e o valor crítico . . . 4

4.2 O teste rejeita 5% quando deveria rejeitar 5%? . . . 5

4.3 E quando a série é estacionária, ele percebe? . . . 5

5 O teste de Ljung-Box (`Estatistica.mqh`) 6

6 Verossimilhança, AIC e a escolha das ordens 7

7 Hannan-Rissanen (`Arima.mqh`) 8

8 GARCH(1,1) (`Garch.mqh`) 8

8.1 As fórmulas 8

8.2 O otimizador acha mesmo o máximo? 9

8.3 Recupera parâmetros conhecidos? 9

9 Previsão e percentil 10

10 Outras conferências já feitas, e onde estão 10

11 O que estes testes provam, e o que não provam 11

12 Perguntas que a banca pode fazer 12

1 Como ler este relatório

1.1 O que é "dar igual" num computador

Um número real no computador (tipo `double`) guarda cerca de 16 algarismos significativos. Duas contas matematicamente iguais, feitas em ordem diferente, podem discordar no último algarismo. A menor diferença relativa que o `double` consegue distinguir perto de 1 é $2{,}2 \times 10^{-16}$ (a "precisão da máquina").

Por isso, num teste de identidade, **diferença da ordem de 10^{-14} a 10^{-17} é igualdade**: é o arredondamento, não a fórmula. Diferença *zero* aparece quando a conta independente faz exatamente as mesmas operações na mesma ordem. Um erro de fórmula (um sinal trocado, um índice deslocado, um n no lugar de $n-1$) apareceria como diferença de 10^{-3} ou maior.

As diferenças são relativas (divididas pelo tamanho do número) quando o número pode ser grande, e absolutas quando é uma probabilidade, uma contagem ou um valor perto de 1.

1.2 O que é "dar perto" numa simulação

Quando o teste é "o ADF a 5% rejeita cerca de 5% das vezes", o número observado nunca é exatamente 5,0%, porque as séries são sorteadas. Com R simulações, a taxa observada oscila em torno da verdadeira com desvio $\sqrt{p(1-p)/R}$. Para $p = 5\%$ e $R = 5000$, isso dá 0,3 ponto percentual. Uma taxa entre 4,4% e 5,6% (dois desvios) é compatível com 5%.

Todas as simulações usam semente fixa. O programa foi rodado duas vezes e deu os mesmos números nas duas.

1.3 De onde vêm os dados reais

Onde o teste precisa de dados reais, usa a mesma janela da validação numérica de 05/10/2026: os 500 fechamentos diários da `PETR4_OFICIAL` (B3) até 18/09/2026, em log-preço. Os resíduos reais são os do ARIMA(1,1,1) ajustado a essa janela.

2 As identidades, todas de uma vez

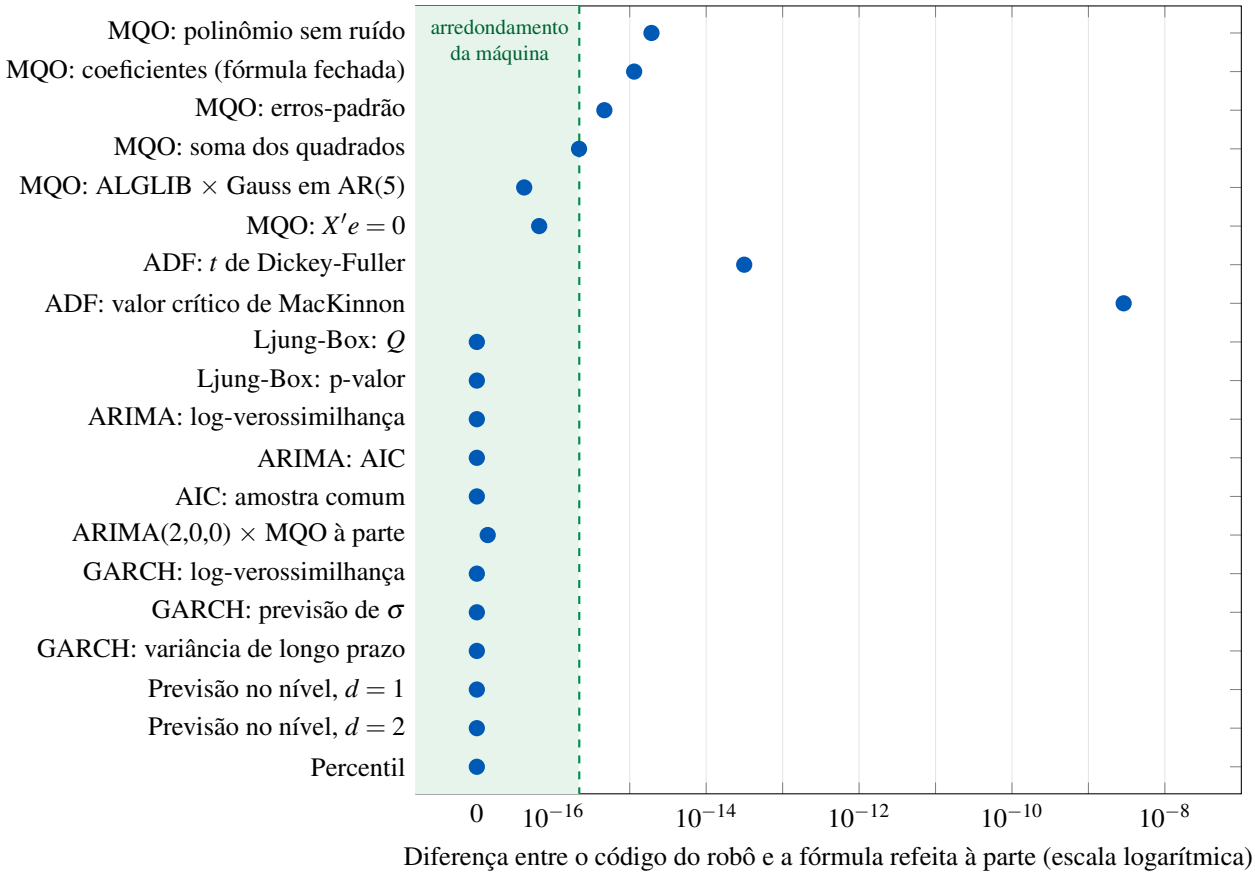


Figura 1: As 20 identidades. Cada ponto é a diferença entre o número do robô e o refeito à parte. Ponto em 0 é igualdade exata. A faixa verde vai até a precisão da máquina ($2{,}2 \times 10^{-16}$). O único ponto fora dela, o do valor crítico, é explicado na seção 4: ele é o próprio termo b_1/T da fórmula, não erro.

A Figura 1 resume tudo. A Tabela 1 diz o que cada linha compara. As seções seguintes explicam cada grupo, e nelas estão também as simulações.

3 Mínimos quadrados (`Numerico.mqh`)

O MQO é a peça mais usada do robô: está no ADF, nos dois estágios do Hannan-Rissanen e na escolha das ordens. Se ele errasse, tudo o resto erraria junto. Ele foi testado de quatro jeitos diferentes (A1 a A6 na tabela).

Com dados sem ruído (A1), a resposta certa é conhecida de antemão: 2, 3 e $-0{,}5$. O código acha esses três números, com diferença de $1{,}9 \times 10^{-15}$.

Com a fórmula do livro (A2 a A4). Na regressão simples (uma variável e a constante), o coeficiente tem fórmula fechada, $b = \sum(x-\bar{x})(y-\bar{y}) / \sum(x-\bar{x})^2$, e o erro-padrão também, $\mathrm{ep}(b) = \sqrt{s^2 / \sum(x-\bar{x})^2}$ com $s^2 = \mathrm{SQR}/(n-2)$. Aplicadas aos retornos reais da PETR4 (retorno de hoje contra o de ontem), dão os mesmos números que o código, que resolve o sistema matricial pela ALGLIB.

Com outro algoritmo (A5). Num AR(5) com constante (seis coeficientes), a solução da ALGLIB foi comparada com uma eliminação de Gauss escrita no próprio programa de teste, sem ALGLIB. Dois caminhos diferentes, mesmo resultado.

Com a propriedade que define o MQO (A6). O MQO é, por definição, o coeficiente que deixa o resíduo

Tabela 1: O que cada identidade compara.

	Código do robô *contra* conta refeita à parte	Diferença
A1	`MQO` em $y = 2 + 3x - 0{,}5x^2$ sem ruído *contra* os coeficientes verdadeiros	$1{,}9 \times 10^{-15}$
A2	`MQO` na regressão do retorno de hoje no de ontem *contra* $b = S_{xy}/S_{xx}$, $a = \bar{y} - b\bar{x}$	$1{,}1 \times 10^{-15}$
A3	erros-padrão do `MQO` *contra* as fórmulas fechadas da regressão simples	$4{,}7 \times 10^{-16}$
A4	soma dos quadrados dos resíduos *contra* a soma refeita	$2{,}2 \times 10^{-16}$
A5	`MQO` (ALGLIB) num AR(5) com constante *contra* eliminação de Gauss escrita à parte	$4{,}2 \times 10^{-17}$
A6	resíduo do `MQO` *contra* a condição $X'e = 0$ (cosseno entre cada coluna e o resíduo)	$6{,}5 \times 10^{-17}$
B1	`RegressaoADF` sem defasagens *contra* o t da regressão $\Delta y_t = a + \gamma y_{t-1}$	$3{,}1 \times 10^{-14}$
B2	`ValorCriticoADF5` com $T = 10^9$ *contra* o $-2{,}86154$ da tabela de MacKinnon	$2{,}9 \times 10^{-9}$
C1	`TesteLjungBox` *contra* $Q = n(n+2)\sum r_k^2/(n-k)$ com autocorrelação escrita à parte	0
C2	p-valor do `TesteLjungBox` *contra* a qui-quadrado com $10 - 2$ graus de liberdade	0
D1	`LogVerossimilhanca` do ARIMA *contra* $-\frac{n}{2}[\ln 2\pi + \ln \hat{\sigma}^2 + 1]$	0
D2	`AIC` do ARIMA *contra* $-2\ln L + 2k$, $k = p + q + 1 + 1$	0
D3	número de observações: maior menos menor entre os 25 candidatos da grade	0
E1	`CArima` ARIMA(2,0,0) *contra* MQO com eliminação de Gauss escrita à parte	$1{,}4 \times 10^{-17}$
F1	`LogVerossimilhanca` do GARCH *contra* a recursão $\sigma_t^2 = \omega + \alpha e_{t-1}^2 + \beta\sigma_{t-1}^2$	0
F2	`VolatilidadeProximoPeriodo` *contra* $\sqrt{\omega + \alpha e_N^2 + \beta\sigma_N^2}$	0
F3	`VarianciaLongoPrazo` *contra* $\omega/(1-\alpha-\beta)$	0
G1	`PreverNivel` com $d = 1$ *contra* $y_N + \hat{z}_{N+1}$	0
G2	`PreverNivel` com $d = 2$ *contra* $2y_N - y_{N-1} + \hat{z}_{N+1}$	0
G3	`Percentil` 70 de $\{1, \ldots, 10\}$ *contra* $1 + 0{,}7 \times 9 = 7{,}3$	0

Fonte: `dados/verificacao/testes_formulas_2026-10-30/identidades.csv`.

sem correlação com cada coluna de X ($X'e = 0$, as "equações normais"). O cosseno entre cada coluna e o resíduo deu $6{,}5 \times 10^{-17}$, ou seja, zero.

Os erros-padrão, em média. A identidade A3 confere a fórmula, mas não diz se o erro-padrão *funciona*. Para isso: em 2000 regressões simuladas com coeficiente verdadeiro 0,5, montou-se o intervalo de 95% ($b \pm t_{0{,}975}$ ep) com o erro-padrão do código. O intervalo cobriu o valor verdadeiro em 95,3% das vezes (desvio da simulação: 0,5 ponto). É o esperado. **passou**

4 O teste ADF (`Estatistica.mqh`, `SelecaoModelo.mqh`)

O ADF decide o d do ARIMA, e o d decide o que o modelo enxerga (preço ou retorno). Testou-se a conta, o valor crítico, e o comportamento do teste inteiro.

4.1 A conta e o valor crítico

Sem defasagens, o ADF é só o teste t do coeficiente γ na regressão $\Delta y_t = a + \gamma y_{t-1} + \varepsilon_t$. Refeito pela fórmula fechada da regressão simples nos log-preços reais, deu o mesmo número que a função do robô (B1, diferença $3{,}1 \times 10^{-14}$).

O valor crítico vem da fórmula de MacKinnon (2010), $b_0 + b_1/T + b_2/T^2 + b_3/T^3$. Quando T cresce sem limite, ela tem de dar $b_0 = -2{,}86154$, que é o número da tabela dele para o caso com constante. Com $T = 10^9$ o código dá $-2{,}86154$ mais $2{,}9 \times 10^{-9}$, e essa sobra é exatamente b_1/T com $b_1 = -2{,}8903$: a fórmula está certa e ainda não chegou ao limite. Na janela do robô (T por volta de 225 a 239) o valor crítico fica perto de $-2{,}874$ (com $T = 225$).

4.2 O teste rejeita 5% quando deveria rejeitar 5%?

Esse é o teste mais importante de um teste estatístico: o *tamanho*. Foram gerados 5000 passeios aleatórios de 240 pontos, que têm raiz unitária por construção. Um ADF a 5% bem implementado deve rejeitar a raiz unitária em cerca de 5% deles, por puro acaso.

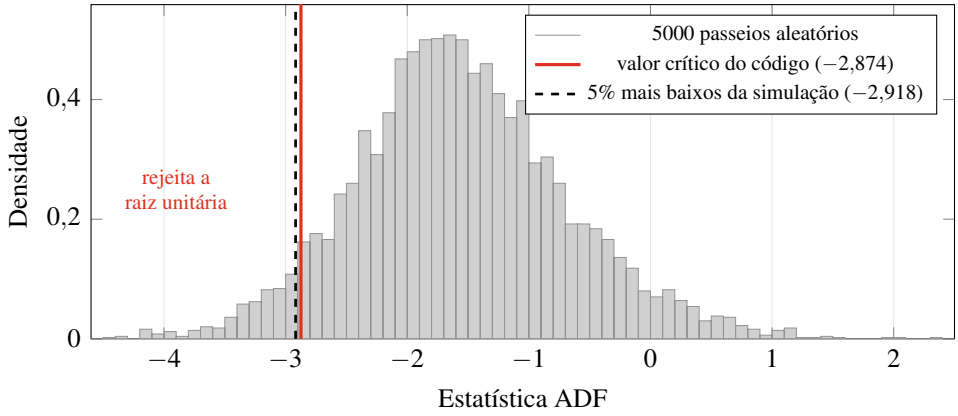


Figura 2: Distribuição da estatística ADF quando a série tem raiz unitária. À esquerda da linha vermelha o teste rejeita. A distribuição não é a normal (centrada em zero): é a de Dickey-Fuller, deslocada para a esquerda, e é por isso que o valor crítico é −2,87 e não −1,65.

O resultado está na Figura 2. Com a escolha de defasagens pelo AIC, exatamente como o robô faz, o teste rejeitou 5,6% dos passeios. Fixando zero defasagens, rejeitou 5,0%. O desvio da simulação é 0,3 ponto. **passou**

O 5,6% está no limite

5,6% fica a quase dois desvios de 5%, e o quantil de 5% da simulação (−2,918) está um pouco à esquerda do valor crítico. Sem escolher defasagens, o teste acerta os 5% em cheio. A diferença vem da escolha das defasagens pelo AIC em amostra pequena, que torna o teste um pouco mais propenso a rejeitar. Não é erro de fórmula: a conta (B1) e o valor crítico (B2) estão certos. Para o robô, o efeito é que, numa série com raiz unitária, ele escolheria $d = 0$ em cerca de 5,6% das janelas em vez de 5%.

4.3 E quando a série é estacionária, ele percebe?

O outro lado é o *poder*: a chance de rejeitar quando a série *não* tem raiz unitária. Foram geradas 2000 séries AR(1), $y_t = \phi y_{t-1} + \varepsilon_t$, para cada ϕ (Figura 3). Com $\phi = 0{,}80$ a série volta depressa à média e o teste percebe quase sempre (99,5%). Com $\phi = 0{,}98$ ela é quase um passeio aleatório, e em 240 pontos o teste só percebe em 12,3% das vezes. É o comportamento conhecido do ADF: pouco poder perto de $\phi = 1$.

O *d* escolhido. O robô não usa o ADF sozinho: ele aplica o teste, diferencia se não rejeitar, e testa de novo (`DeterminarOrdemD`). À direita na Figura 3, o d certo foi escolhido em 99,8% das séries I(0), 92,8% das I(1) e 90,0% das I(2). O log-preço da PETR4 é o caso I(1). **passou**

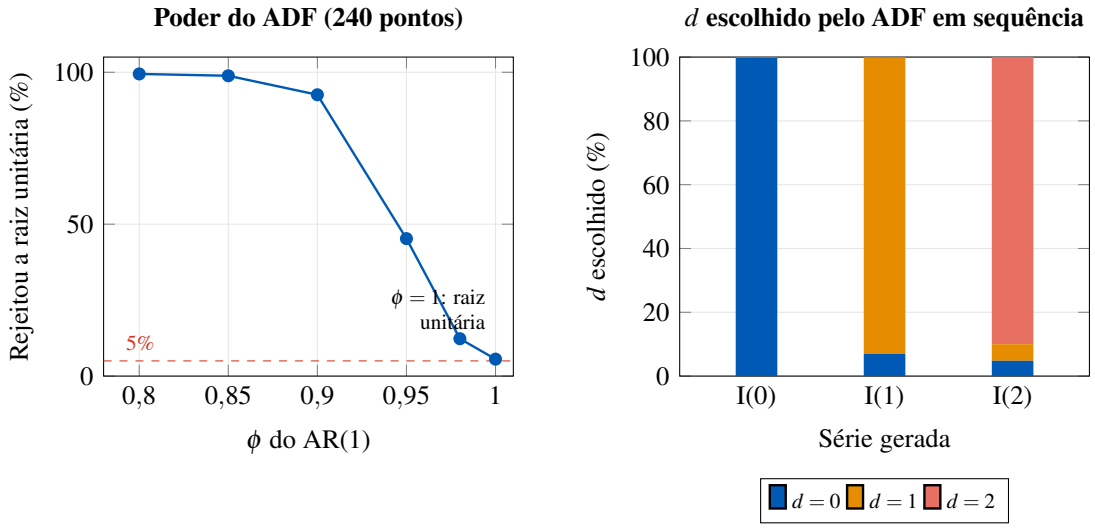


Figura 3: À esquerda, a chance de o ADF rejeitar a raiz unitária conforme a série se aproxima de um passeio aleatório ($\phi \to 1$); em $\phi = 1$ é o tamanho do teste. À direita, 1000 séries de cada tipo passadas pela função `DeterminarOrdemD` do robô, com d máximo 2, como no robô.

5 O teste de Ljung-Box (`Estatistica.mqh`)

Onde ele é usado

O EA *não* usa o Ljung-Box para decidir nada: ele aparece nos diagnósticos e na validação numérica (por exemplo, "sobra autocorrelação nos resíduos do ARIMA(1,1,1)"). O que está nesta seção afeta essas conclusões, não as operações do robô.

A conta foi refeita nos resíduos reais do ARIMA(1,1,1): autocorrelações escritas à parte, $Q = n(n+2)\sum_{k=1}^{10} r_k^2/(n-k)$, e p-valor pela qui-quadrado com $10-2$ graus de liberdade. Os dois números saíram iguais aos do código, com diferença zero (C1 e C2). **passou**

O tamanho. Em 5000 séries de ruído branco de 239 pontos, o Q do código deve seguir uma qui-quadrado com 10 graus de liberdade, e o teste deve rejeitar cerca de 5%. A Figura 4 mostra que o histograma acompanha a curva teórica. A rejeição foi 5,5% (desvio 0,3). **passou**

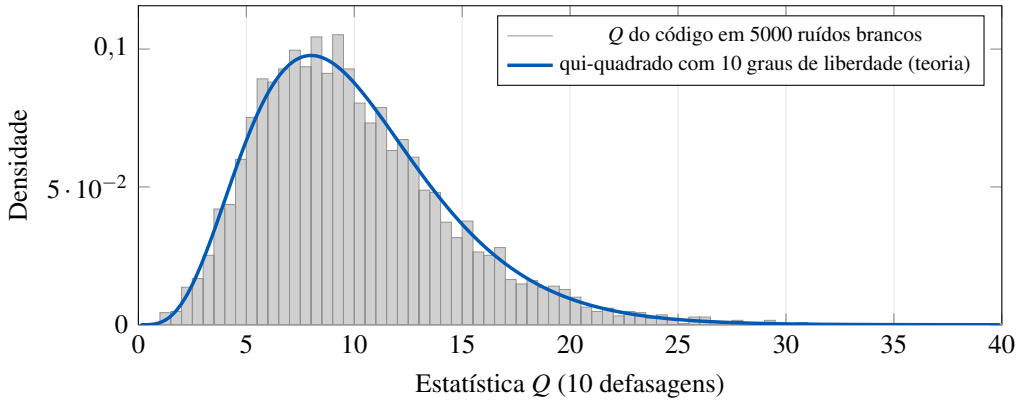


Figura 4: Sem autocorrelação na série, o Q calculado pelo código segue a distribuição teórica.

O desconto de graus de liberdade. Quando o Ljung-Box é aplicado aos resíduos de um ARMA(p,q) ajustado, a regra usual desconta $p+q$ graus de liberdade (a qui-quadrado passa de 10 para 8), porque o ajuste já "gastou" parte da autocorrelação. O código faz esse desconto. Para ver o efeito, foram ajustados ARMA(1,1) pelo código do robô em 2000 séries ARMA(1,1) simuladas, e os resíduos foram testados das

duas formas.

Achado: com o desconto, o teste rejeita demais

Com 8 graus de liberdade (a regra, e o que o código faz), o teste rejeitou 7,9% dos resíduos. Sem o desconto (10 graus), 3,8%. O modelo ajustado é o certo nas duas contas, então o ideal seria 5%: o desconto corrige demais, e a versão sem desconto corrige de menos.
A causa **não foi confirmada**. A explicação mais provável (estimativa) é que a regra de $p+q$ graus foi deduzida para coeficientes estimados por máxima verossimilhança, e o Hannan-Rissanen é um pouco menos preciso (seção 7 e o relatório do Hannan-Rissanen), deixando um pouco mais de autocorrelação nos resíduos. Consequência prática: um p-valor do Ljung-Box perto de 0,05 nos resíduos do robô não é evidência forte. Um p-valor bem menor, como o 0,009 da validação de 05/10, continua sendo.

O poder. Numa série AR(1) com $\phi = 0{,}2$ (autocorrelação fraca, do tamanho das que aparecem em retornos), o teste com 10 defasagens percebeu a autocorrelação em 51,1% das 2000 séries de 239 pontos.

6 Verossimilhança, AIC e a escolha das ordens

As fórmulas (D1 e D2). Com resíduos gaussianos e $\hat{\sigma}^2 = \mathrm{SQR}/n$, a log-verossimilhança do ARIMA é $\ln L = -\frac{n}{2}[\ln(2\pi) + \ln\hat{\sigma}^2 + 1]$, e o AIC é $-2\ln L + 2k$, com k contando p, q, a constante e a variância. Refeitas a partir dos resíduos que o próprio `CArima` devolve, deram diferença zero.

A amostra comum (D3). O AIC só compara modelos de forma justa se todos somarem a verossimilhança sobre as mesmas observações (é a correção descrita no `CLAUDE.md`, "AIC sempre sobre amostra comum"). Os 25 candidatos da grade, de ARIMA(0,1,0) a ARIMA(4,1,4), usaram todos 495 observações. Diferença entre o maior e o menor: zero. **passou**

A grade escolhe a ordem certa? Foram geradas 500 séries cujo retorno é um AR(1) com $\phi = 0{,}5$ e 500 passeios aleatórios puros (retorno sem memória, ordem verdadeira (0,1,0)). Cada uma passou pela `BuscarMelhorPQ` do robô, com p e q de 0 a 4 (Figura 5).

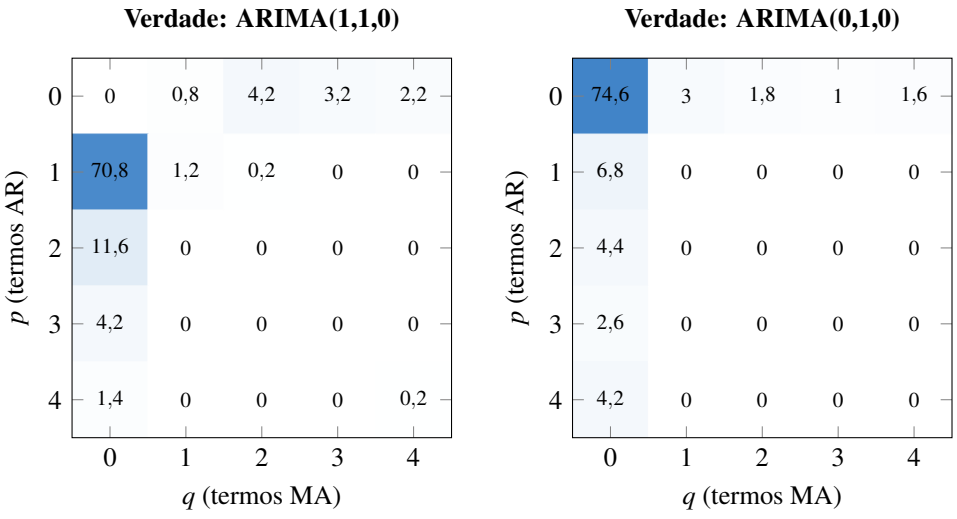


Figura 5: Porcentagem das vezes em que a grade do AIC escolheu cada (p,q), em séries de 240 pontos com ordem verdadeira conhecida.

A ordem verdadeira foi a mais escolhida nos dois casos: 70,8% no AR(1) e 74,6% no passeio. Quando erra, o AIC em geral escolhe um modelo com *mais* termos que o verdadeiro (no AR(1), o segundo mais escolhido foi o ARIMA(2,1,0)). Isso é propriedade conhecida do AIC, que penaliza pouco cada parâmetro,

e não erro do código. Vale lembrar ao ler o Cap. III: quando a grade escolhe uma ordem alta na PETR4, parte disso pode ser esse excesso. **passou**

7 Hannan-Rissanen (`Arima.mqh`)

O relatório próprio (`ia/relatorios/hannan-rissanen/`) explica o método e refaz dois pregões reais: os coeficientes e a previsão saíram **idênticos** aos do traço do robô, com diferença zero (`dados/resultados/hannan_rissanen_2026-10-30/resumo.txt`). Aqui ficam dois testes que faltavam.

Sem termo MA, é MQO (E1). Quando $q = 0$ o Hannan-Rissanen não precisa do estágio 1, e o ARIMA(p,0,0) é uma regressão comum. O ARIMA(2,0,0) do `CArima` nos retornos reais deu os mesmos três coeficientes que uma eliminação de Gauss escrita à parte (diferença $1{,}4 \times 10^{-17}$).

Recupera coeficientes conhecidos? Para cada valor verdadeiro de $-0{,}6$ a $0{,}6$ foram simuladas 500 séries de 239 pontos, AR(1) e MA(1), e ajustadas pelo `CArima`. A Figura 6 mostra a média das estimativas e o desvio (barra). Os pontos caem na diagonal: em média o método acerta. O desvio é maior no MA do que no AR, porque o MA depende dos erros estimados no estágio 1. **passou**

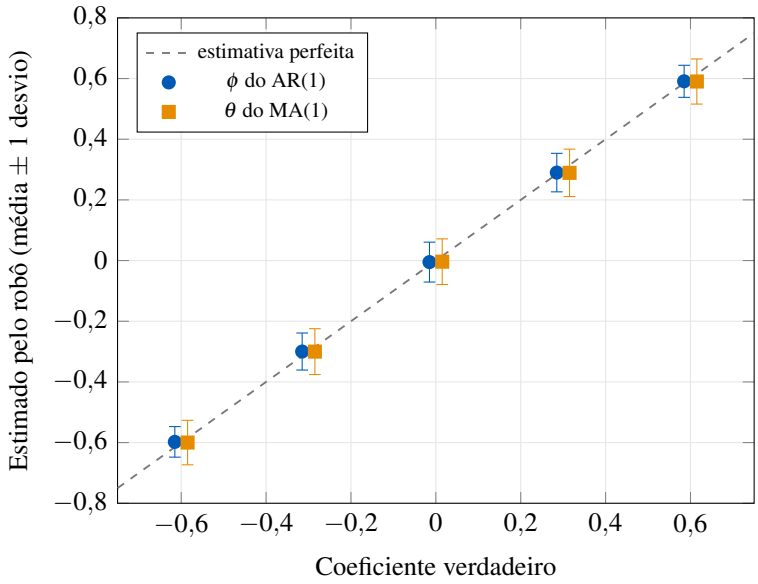


Figura 6: Recuperação de coeficientes conhecidos pelo `CArima`, 500 séries de 239 pontos por ponto. Os pontos foram afastados um pouco na horizontal para não se sobreporem.

8 GARCH(1,1) (`Garch.mqh`)

8.1 As fórmulas

A log-verossimilhança do GARCH foi refeita à parte: variância inicial igual à média de e^2, depois a recursão $\sigma_t^2 = \omega + \alpha e_{t-1}^2 + \beta \sigma_{t-1}^2$ e a soma de $-\frac{1}{2}[\ln 2\pi + \ln \sigma_t^2 + e_t^2/\sigma_t^2]$. Com os parâmetros que o robô estimou nos resíduos reais, deu o mesmo número (F1). A previsão de um passo $\sqrt{\omega + \alpha e_N^2 + \beta \sigma_N^2}$ (F2), que é o número comparado com o limiar, e a variância de longo prazo $\omega/(1-\alpha-\beta)$ (F3) também. Diferença zero nas três. **passou**

Os parâmetros estimados nessa janela foram $\omega = 7{,}363 \times 10^{-5}$, $\alpha = 0{,}1264$ e $\beta = 0{,}5878$, com $\ln L = 1359{,}525$. São os mesmos da validação de 05/10/2026 (`dados/verificacao/validacao_numerica_2026-10-05.txt`): o código não mudou de comportamento desde então.

8.2 O otimizador acha mesmo o máximo?

As identidades conferem a *conta* da verossimilhança. Falta conferir se o otimizador (MinBLEIC, da ALGLIB) devolve o ponto mais alto dela. Dois testes:

Grade. Com ω fixo, calculou-se a verossimilhança em 1681 combinações de α e β em volta do ponto estimado, de 0,005 em 0,005 (Figura 7). Nenhuma passou o ponto do robô: 0 de 1681. O vizinho mais alto ficou $-8{,}6 \times 10^{-3}$ abaixo.

Perturbação. Mexendo um parâmetro de cada vez (1% em ω, 0,001 em α e em β), para mais e para menos, a verossimilhança sempre caiu. A menor queda foi $-7{,}2 \times 10^{-4}$. Num máximo, a verossimilhança cai para os dois lados, e é o que se vê. **passou**

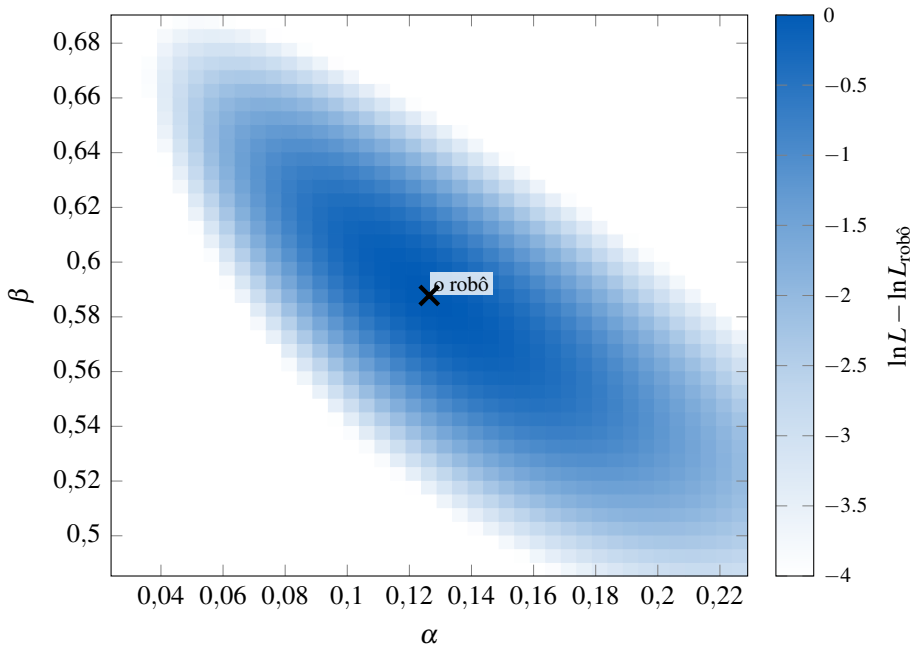


Figura 7: Log-verossimilhança do GARCH(1,1) nos resíduos reais, em volta do ponto estimado pelo robô (ω fixo). Quanto mais azul, mais alto. O ponto do robô está no topo: nenhuma das 1681 combinações fica acima dele. Abaixo de -4 fica branco. A faixa azul inclinada mostra que α e β se compensam: subir um e descer o outro muda pouco a verossimilhança, e é por isso que estimá-los separadamente é difícil (seção 8.3).

8.3 Recupera parâmetros conhecidos?

Foram simuladas séries GARCH(1,1) com $\omega = 10^{-5}$, $\alpha = 0{,}09$ e $\beta = 0{,}89$ (os mesmos da validação de 05/10/2026), 300 com 240 pontos e 300 com 1000 pontos, e cada uma foi ajustada pelo `CGarch` (Figura 8).

Com 1000 pontos as estimativas se juntam em volta da estrela: α médio 0,089 (desvio 0,021) e β médio 0,884 (desvio 0,027). A implementação recupera os parâmetros. **passou**

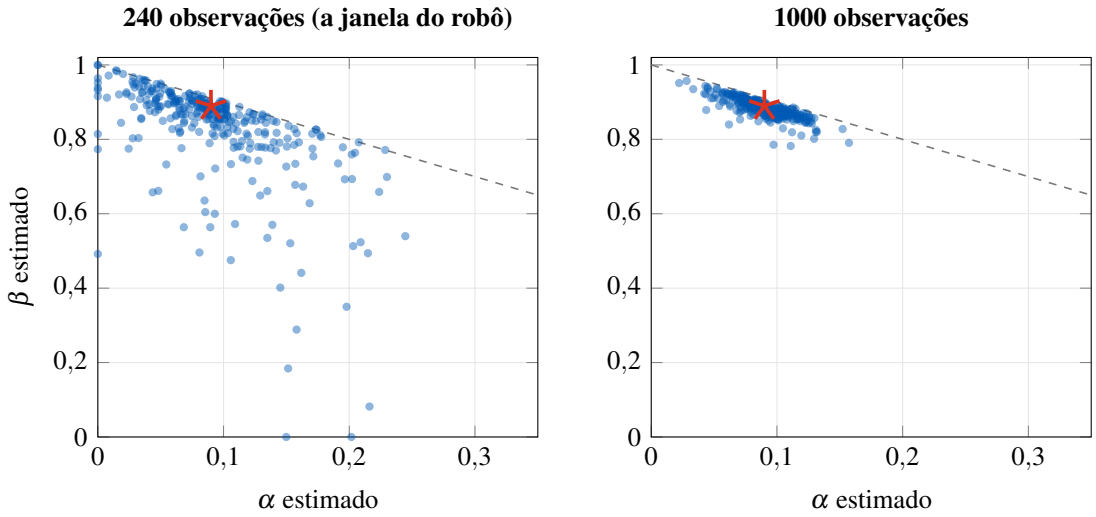


Figura 8: Cada ponto é um ajuste do `CGarch` numa série simulada. A estrela é o valor verdadeiro. A linha tracejada é o limite $\alpha + \beta < 1$, que o otimizador respeita (nenhum ponto passa dela).

Achado: com 240 pontos, o β é incerto

Com 240 pontos, o α médio ainda acerta (0,092), mas o β médio cai para 0,824 com desvio de 0,142: a nuvem se espalha para baixo, e 3,3% dos ajustes encostaram no limite $\alpha + \beta = 1$. É uma propriedade do GARCH, não do código (com mais dados a mesma função acerta): a persistência da volatilidade é difícil de medir em menos de um ano de pregões.
Para o robô, isso quer dizer que o α e o β estimados em cada janela de 240 pregões oscilam bastante de uma janela para outra por puro erro de estimação. O que o robô usa é a volatilidade prevista comparada a um percentil da própria janela, e não os parâmetros em si. Se essa comparação é menos sensível ao erro de β é algo **não testado** aqui.

9 Previsão e percentil

A previsão do ARIMA é feita no espaço diferenciado ($\hat{z}_{N+1}$, o retorno) e depois volta ao nível. Com $d = 1$, $\hat{y}_{N+1} = y_N + \hat{z}_{N+1}$. Com $d = 2$, $\hat{y}_{N+1} = 2y_N - y_{N-1} + \hat{z}_{N+1}$. As duas saíram iguais à função `PreverNivel`, com diferença zero (G1 e G2).

O percentil, que define o limiar do filtro, usa interpolação linear entre os valores ordenados. Para os números de 1 a 10, o percentil 70 fica na posição $0,7 \times 9 = 6,3$ (contando do zero), ou seja, $7 + 0,3 \times (8 - 7) = 7,3$. Foi o que o código deu (G3). **passou**

10 Outras conferências já feitas, e onde estão

Este relatório se soma a conferências anteriores, cada uma com o seu arquivo:

- **Recuperação de parâmetros em uma série sintética** (05/10/2026): ARMA(1,1), AR(2) e GARCH(1,1) recuperados perto do verdadeiro, e o ADF distinguindo passeio aleatório de série estacionária. `dados/verificacao/validacao_numerica_2026-10-05.txt`.
- **Hannan-Rissanen contra o traço do robô** (30/10/2026): dois pregões reais refeitos fora do robô, constante, θ e previsão idênticos. `dados/resultados/hannan_rissanen_2026-10-30/resumo.txt`.

- **Teste de regressão** (18/10/2026): `ferramentas/scripts/regressao_codigo.sh`, 13 casos que passam por todos os caminhos dos dois EAs, comparados byte a byte antes e depois da limpeza do código. Não foi rodado de novo nesta sessão, porque o código do robô não mudou.
- **GARCH com *t* de Student e máxima verossimilhança no ARIMA** (13 e 15/10/2026): as duas opções experimentais têm validação própria em `dados/verificacao/`. Não fazem parte do TG nem deste relatório.

11 O que estes testes provam, e o que não provam

Provam que cada função testada calcula a fórmula que diz calcular: os mesmos números saem quando a conta é refeita por outro caminho (fórmula fechada, outro algoritmo, propriedade que define o método). E provam que, em dados com resposta conhecida, o código se comporta como a teoria manda: testes com o tamanho certo, coeficientes recuperados, máximo encontrado.

Não provam três coisas.

- As contas independentes foram escritas a partir da mesma leitura das fórmulas. Se a fórmula tivesse sido mal lida dos livros, o erro estaria dos dois lados. O que protege contra isso são os testes que não dependem da leitura: os valores verdadeiros das simulações, o valor da tabela de MacKinnon e a distribuição qui-quadrado da biblioteca estatística do MetaTrader. Uma conferência a mais, possível e fora do repositório, seria rodar a mesma série num software estatístico estabelecido e comparar os coeficientes.
- Não testam a parte de execução do robô (ordens, posição, volume). Essa parte é coberta pelo teste de regressão e pelo traço por barra.
- Não testam se as escolhas de método são as melhores (por exemplo, AIC em vez de BIC, ou Hannan-Rissanen em vez de máxima verossimilhança). Testam que as escolhas feitas estão implementadas corretamente.

12 Perguntas que a banca pode fazer

Se a banca perguntar

"Como vocês sabem que o ARIMA e o GARCH em MQL5 estão certos, se não usaram uma biblioteca pronta?"
Cada função foi testada de duas formas. Primeiro, refazendo a conta por outro caminho e conferindo que o resultado é o mesmo até o arredondamento do computador. Segundo, com séries simuladas de propriedades conhecidas: o código recupera os coeficientes verdadeiros e os testes estatísticos rejeitam na proporção certa.

"O otimizador do GARCH pode parar num ponto que não é o máximo?"
Na janela real testada, nenhuma das 1681 combinações vizinhas teve verossimilhança maior que a do ponto devolvido, e qualquer perturbação pequena fez a verossimilhança cair.

"Com 240 pregões dá para estimar um GARCH?"
Dá, mas com incerteza grande no β: nas simulações o desvio da estimativa foi 0,142 com 240 pontos, contra 0,027 com 1000. É uma limitação do tamanho da janela, que é escolha do experimento.

"Por que o valor crítico do ADF é $-2{,}87$ e não $-1{,}65$?"
Porque, sob a hipótese de raiz unitária, a estatística não segue a normal: segue a distribuição de Dickey-Fuller, deslocada para a esquerda (Figura 2). Os valores críticos vêm das tabelas de MacKinnon.

Para refazer: `bash ferramentas/scripts/testes_formulas.sh` (cerca de um minuto) e `make` nesta pasta. Código dos testes: `ferramentas/TestesFormulas.mq5`. Dados: `dados/verificacao/testes_formulas_2026-10-30/`.